Анализ и концептуальное моделирование систем

профессор, д.т.н. Смоленцева Т.Е.:
smolenceva@mirea.ru:

ЛЕКЦИЯ 3.
Объектно-ориентированное моделирование.

СОДЕРЖАНИЕ ЛЕКЦИИ:

- Сущность структурного подхода. Базовые принципы структурного подхода
- Сущность объектно-ориентированного подхода
- Базовые принципы объектно-ориентированного подхода: уникальность, классификация, инкапсуляция, наследование, полиморфизм

1. Сущность структурного подхода. Базовые принципы структурного подхода

Принципы структурного похода:

1. **Разделяй и властвуй**. Для решения сложной задачи используется ее разбиение на несколько более простых подзадач.

2. **Иерархическое упорядочивание**. Решение задачи представляется в виде иерархии описаний (функций, потоков данных), где описания вышестоящих уровней детализируются при помощи описаний нижестоящих уровней.

3. **Абстрагирование**. При решении задачи рассматриваются только важные ее аспекты; все незначительное отбрасывается.

4. **Формализация**. Все принятые решения описываются формально.

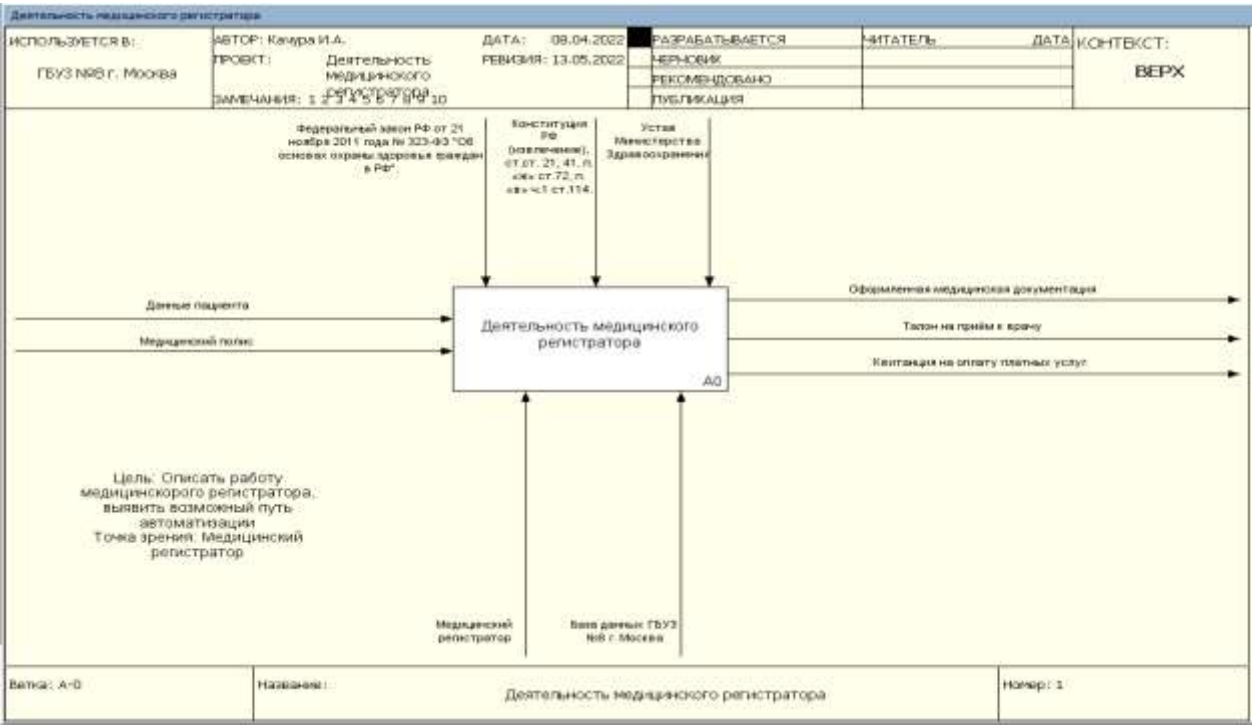
5. **Непротиворечивость**. Никакая часть системы не может противоречить никакой другой. Все должно быть согласованно и находиться в соответствии.

Модель процесса

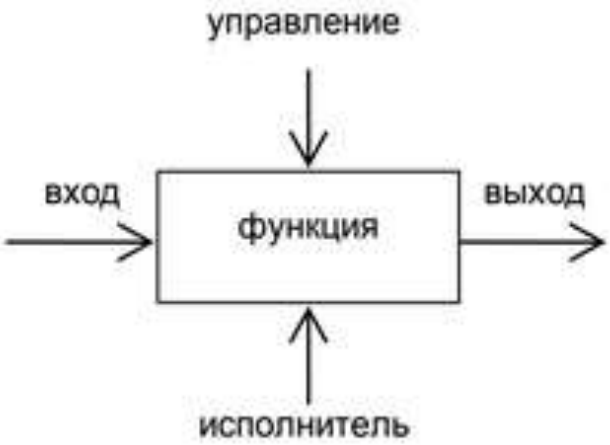
Для структурного подхода характерен *каскадный жизненный цикл* системы, когда каждый последующий этап решения задачи выполняется после полного завершения предыдущего этапа.

Это является следствием принципа *иерархической упорядоченности*.

Система (как и ее описание) представляет собой иерархическую структуру, в которой вышестоящие уровни определяются через нижестоящие. Любую вершину в иерархии можно считать определенной только в том случае, если определены все ее потомки.

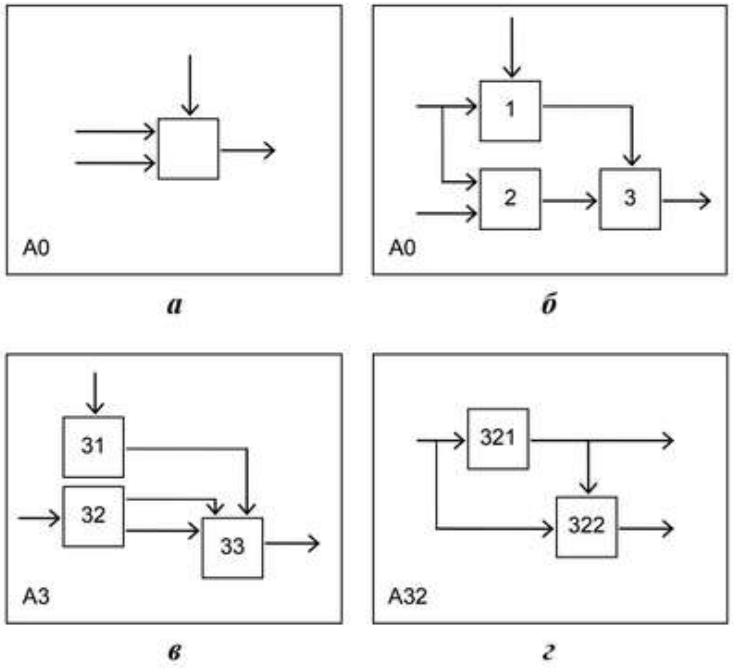


Методология ***SADT*** представляет собой совокупность методов, правил и процедур, предназначенных для построения функциональной модели объекта какой-либо предметной области. Функциональная модель *SADT* отображает функциональную структуру объекта, т.е. производимые им действия и связи между этими действиями.



Графическая нотация диаграмм SADT

Иерархия диаграмм



а *б* *в* *г*

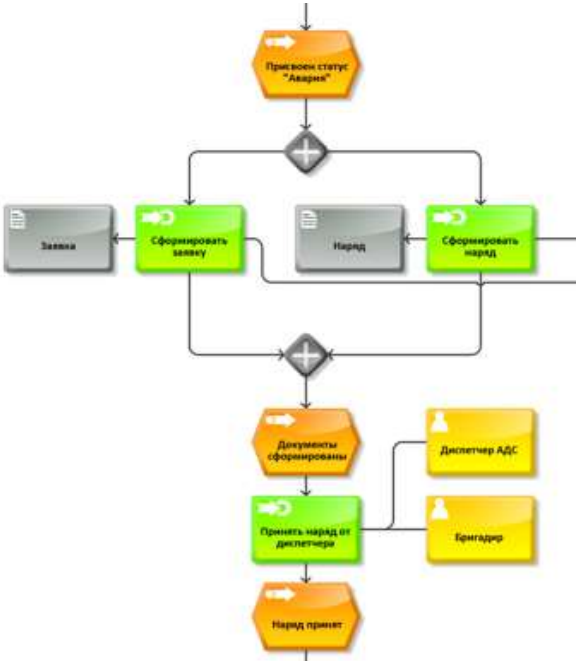
Детализация функций на дочерних диаграммах

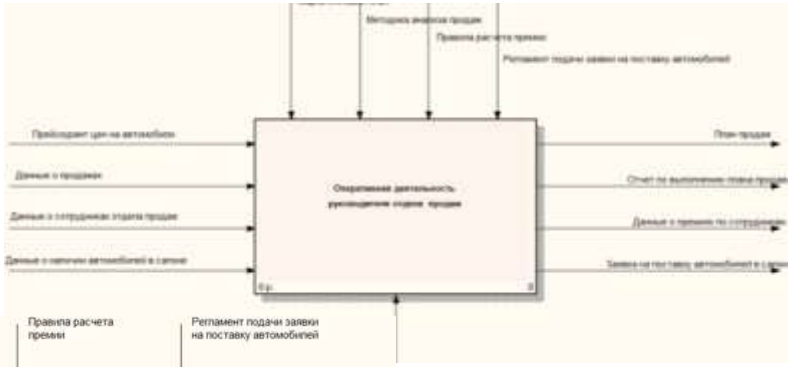
Сначала строится корневая диаграмма, на которой моделируемый объект (вся система в целом или какая-то ее подсистема) представляется в виде одной функции, для которой определяются все входные параметры, выходные величины, управляющая информация, а также исполнители (а).

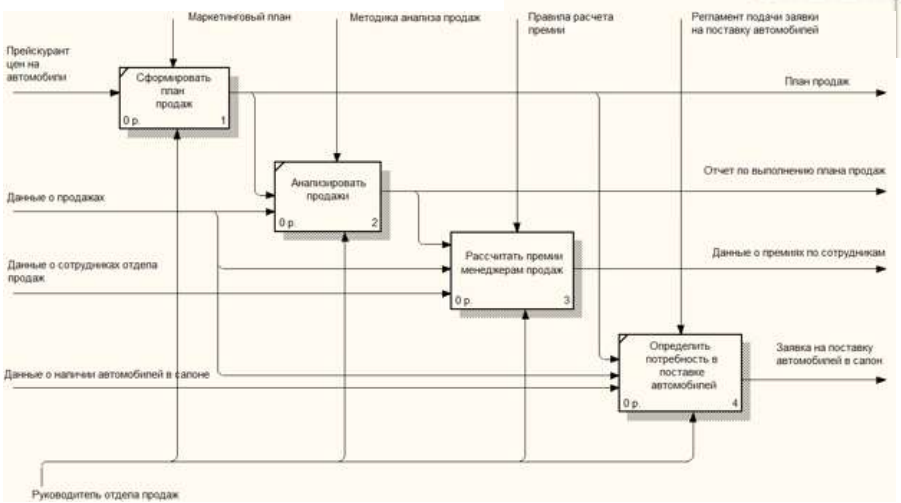
После этого полученная диаграмма детализируется путем разбиения основной функции на несколько подфункций(б).

Далее процесс детализации рекурсивно повторяется для каждой нетривиальной подфункции (в, г).

Примеры реализации:









Сравнение существующих методик

*1. В **функциональных моделях** (DFD-диаграммах потоков данных, SADT-диаграммах) главными структурными компонентами являются **функции** (операции, действия, работы), которые на диаграммах связываются между собой потоками объектов.*

*2. **Достоинством** функциональных моделей является реализация структурного подхода к проектированию ИС по принципу "сверху-вниз", когда каждый функциональный блок может быть декомпозирован на множество подфункций и т.д., выполняя, таким образом, модульное проектирование ИС.*
Для функциональных моделей характерны процедурная строгость декомпозиции ИС и наглядность представления.

*3. При функциональном подходе объектные модели данных в виде **ER-диаграмм "объект — свойство — связь"** разрабатываются отдельно. Для проверки корректности моделирования предметной области между функциональными и объектными моделями устанавливаются взаимно однозначные связи.*

*4. Главный **недостаток** функциональных моделей заключается в том, что процессы и данные существуют отдельно друг от друга — помимо функциональной декомпозиции существует структура данных, находящаяся на втором плане. Кроме того, не ясны условия выполнения процессов обработки информации, которые динамически могут изменяться.*

*5. Перечисленные недостатки функциональных моделей снимаются в **объектно-ориентированных моделях**, в которых главным структурообразующим компонентом выступает **класс объектов с набором функций**, которые могут обращаться к атрибутам этого класса.*

2. Сущность объектно-ориентированного подхода

Первые объектно-ориентированные методы появились в начале 80-х гг. XX в., однако основные результаты были получены спустя 10 лет, когда Айвар Джекобсон, Джим Рембо и Грейди Буч представили свои подходы – *Objectory*, *OMT (Object Modeling Technique* – ***техника объектного моделирования***) и *Booch*.

Эти методы были впоследствии объединены в один, из которого вырос унифицированный язык моделирования –*UML (Unified Modeling Language*).



Суть объектного подхода заключается в объектной декомпозиции, т.е. система представляется в виде совокупности объектов, которые в процессе взаимодействия обмениваются сообщениями. ***Объект*** *– это самостоятельная, самодостаточная сущность, обладающая состоянием, поведением и семантикой.*

Авторы поставили перед собой три главные ***цели****:*

– моделировать системы целиком, от концепции до исполняемого артефакта, с помощью объектно-ориентированных методов;

– решить проблему масштабируемости, которая присуща сложным системам, предназначенным для выполнения ответственных задач;

– создать такой язык моделирования, который может использоваться не только людьми, но и компьютерами.



3. Базовые принципы объектно-ориентированного подхода

Объектно-ориентированному подходу присуще большинство принципов структурного подхода, а именно: *«разделяй и властвуй», абстрагирование, формализация и непротиворечивость*. Кроме того, выделяются следующие специфичные для него принципы:

1.Уникальность. *Каждый объект имеет свой адрес в памяти*, и две объектные переменные считаются равными только в том случае, если они указывают на один и тот же адрес.

2. Классификация. Все *объекты объединяются в классы по принципу сходства структуры, поведения и семантики*.

3. Инкапсуляция. *Совместное «хранение» данных и методов их обработки.* В широком смысле инкапсуляция – это *скрытие реализации за интерфейсом*, т.е. объект обладает внутренней, известной лишь ему структурой и интерфейсом.

4. Наследование. Классы могут объединяться в иерархии наследования. Структура, поведение и семантика объектов наследуются вниз по иерархии.

5. Полиморфизм. Данный принцип означает возможность доступа к нескольким реализациям через один интерфейс. Способом реализации полиморфизма в объектно-ориентированных языках программирования является механизм переопределения унаследованных методов.

Уникальность

Уникальность - это то, что отличает *объект* от других *объектов*.

В машинном представлении под параметром уникальности объекта чаще всего понимается адрес размещения объекта в памяти. Уникальность объекта состоит в том, что всегда можно определить, указывают две ссылки на один и тот же объект или на разные объекты. При этом два объекта могут во всем быть похожими, их образ в памяти может представляться одинаковыми последовательностями байтов, но, тем не менее, их Identity может быть различна.

*распространенной **ошибкой** является понимание уникальности как имени ссылки на объект. Это неверно, т.к. на один объект может указывать несколько ссылок, и ссылки могут менять свои значения (ссылаться на другие объекты).*

Классы

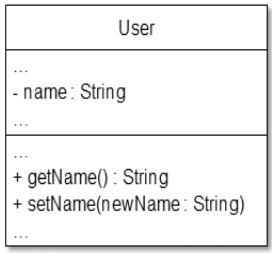
```
public class ClassObject {
    public static void main(String[] args) {
Person person1=new Person();
} }
class Person {
   private String name;
   private int age;
  //конструктор
     public Person (){
       System.out.println("Имя ... ");
   }    //конструктор 1
    public Person (String name){
        this.name=name;
    }    public void setName (String name){
        if (name.isEmpty() ){
            System.out.println("пустое");
        } else
        this.name=name;
    }    public String getName (){
        return name;
    }
```

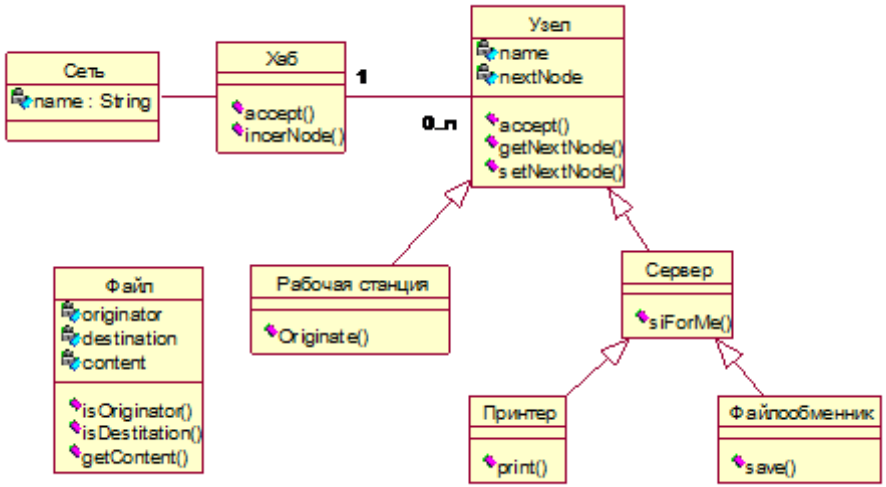
Все объекты одного и того же класса описываются одинаковыми наборами атрибутов.

Объединение объектов в классы позволяет рассмотреть задачу в более общей постановке.

это шаблон поведения объектов определенного типа с заданными параметрами, определяющими состояние. Все экземпляры одного класса (объекты, порожденные от одного класса) имеют один и тот же набор свойств и общее поведение, то есть одинаково реагируют на одинаковые сообщения.







Каждый класс также может иметь специальные методы, которые автоматически вызываются при создании и уничтожении объектов этого класса:

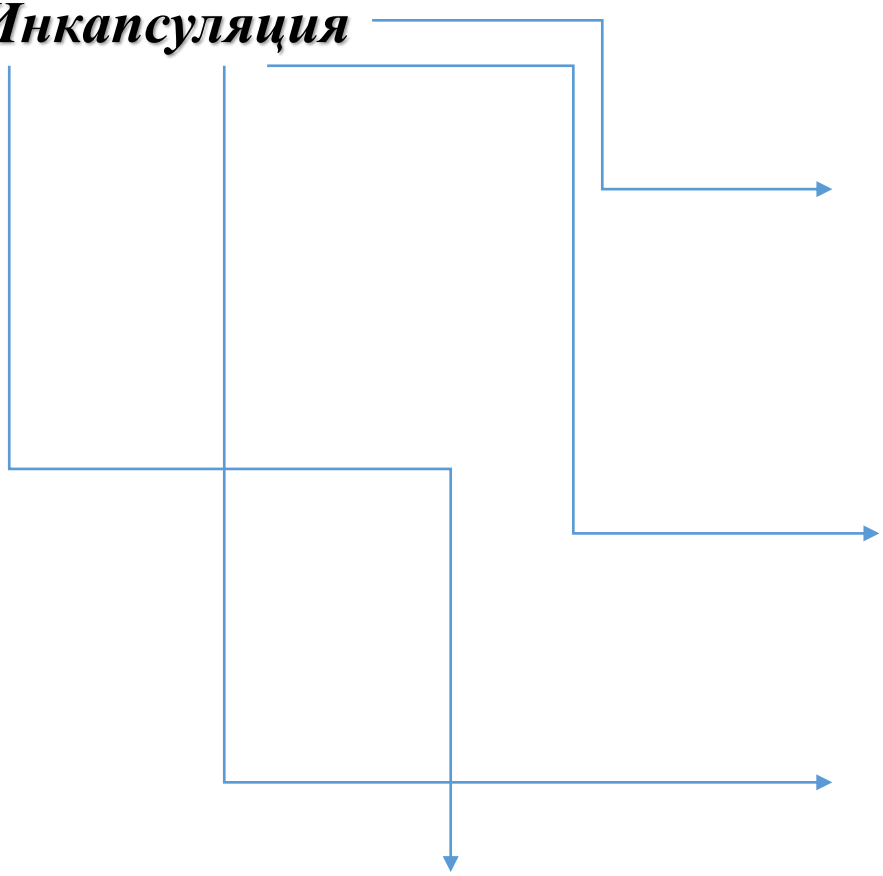
- ***конструктор*** *(constructor) - выполняется при создании объектов ;*
- ***деструктор*** *(destructor) - выполняется при уничтожении объектов.*

```
import java.time.LocalDateTime;
class BankAccount {
   String name;
   LocalDateTime opened;
   double balance;

   public BankAccount() {
      name = "";
      opened = LocalDateTime.now();
      balance = 0.0d;
   }

   public BankAccount(String name, LocalDateTime opened, double
balance) {
      this.name = name;
      this.opened = opened;
      this.balance = balance;
   }
```

Инкапсуляция



сокрытие реализации класса и отделение его внутреннего представления от внешнего (интерфейса)

При использовании объектно-ориентированного подхода не принято применять прямой доступ к свойствам какого-либо класса из методов других классов. Для доступа к свойствам класса принято задействовать специальные методы этого класса для получения и изменения его свойств.

Открытые члены класса составляют внешний интерфейс объекта.

Обеспечение доступа к свойствам класса только через его методы также дает ряд преимуществ. Во-первых, так гораздо проще контролировать корректные значения полей, ведь прямое обращение к свойствам отслеживать невозможно, а значит, им могут присвоить некорректные значения.

Наследование

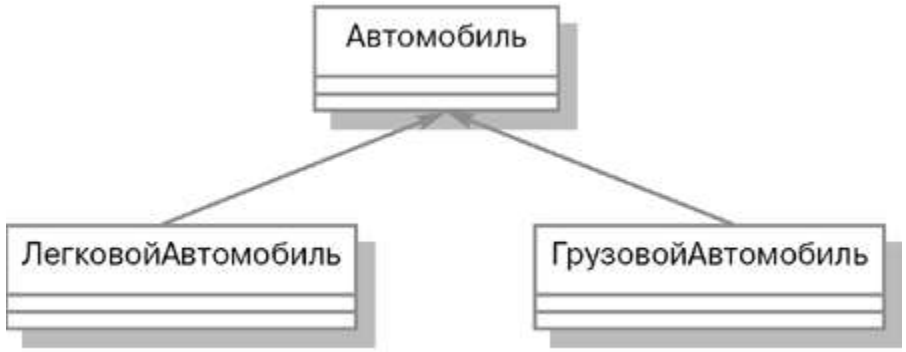
это отношение между классами, при котором класс использует структуру или поведение другого класса (одиночное наследование), или других (множественное наследование) классов

*Наследование вводит иерархию "общее/частное", в которой **подкласс** наследует от одного или нескольких более общих **суперклассов**. Подклассы обычно дополняют или переопределяют унаследованную структуру и поведение.*

```
public class Person1 {
   private String name;
   private String email;

      public static void main(String[] args) {

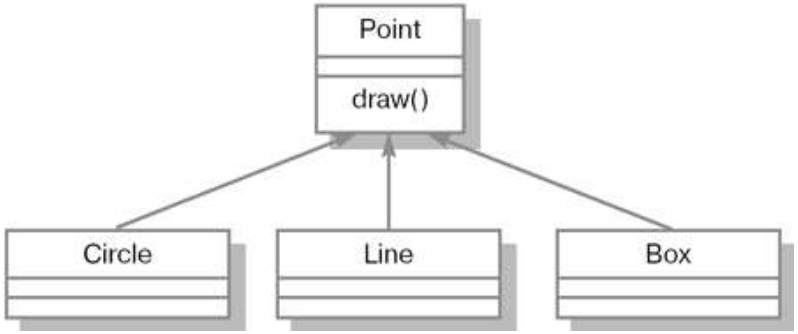
      }
      public class Teacher extends Person1 {
         private String post;
         public void setPost(String post) {
            if (post.isEmpty()) {
               System.out.println("пустое");
            } else
               this.post = post;
         }
         public String getPost() {
            return post;
         }        }
      public class Student extends Person1 {
         private int course;
      }     }
```



Полиморфизм

является одним из фундаментальных понятий в объектно-ориентированном программировании наряду с наследованием и инкапсуляцией. Слово " полиморфизм " греческого происхождения и означает "имеющий много форм".

Полиморфизм (polymorphism) - положение теории типов, согласно которому имена (например, переменных) могут обозначать объекты разных (но имеющих общего родителя) классов. Следовательно, любой объект, обозначаемый полиморфным именем, может по-своему реагировать на некий общий набор операций



Модель процесса

Для объектно-ориентированного подхода характерен *спиральный жизненный цикл* системы. В отличие от структурного подхода, *объектный подход не использует иерархических структур, в которых определение* родителя не имеет смысла без определения всех его потомков.

Вместо этого применяется система объектов, каждый из которых можно анализировать, проектировать, реализовывать и тестировать практически независимо от других. Таким образом, появляется возможность формирования итераций, на каждой из которых выполняются все основные этапы разработки для относительно небольшой группы объектов.

Объектно-ориентированный анализ

В процессе объектно-ориентированного анализа основное внимание уделяется выделению и описанию понятий предметной области, их структуры и связей между ними. Например, в задаче автоматизации библиотеки среди понятий могут присутствовать «библиотека», «издание», «читатель» и др.

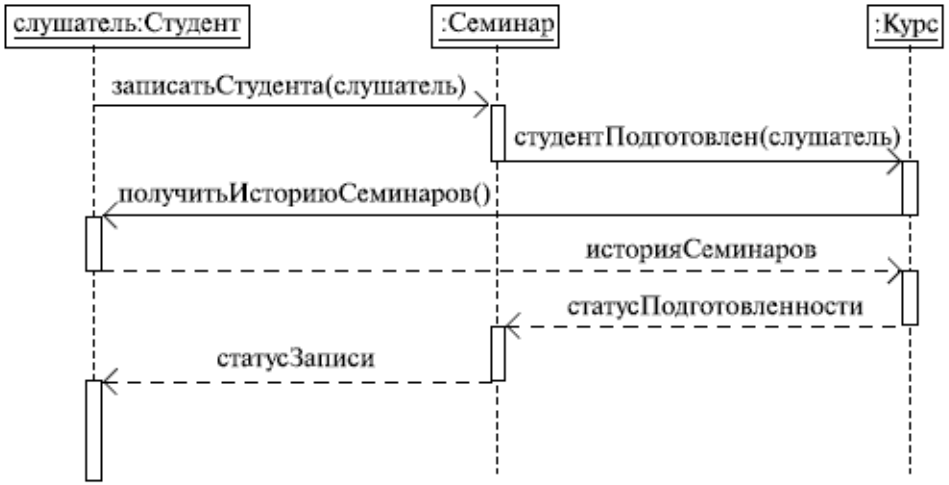
Объектно-ориентированное проектирование

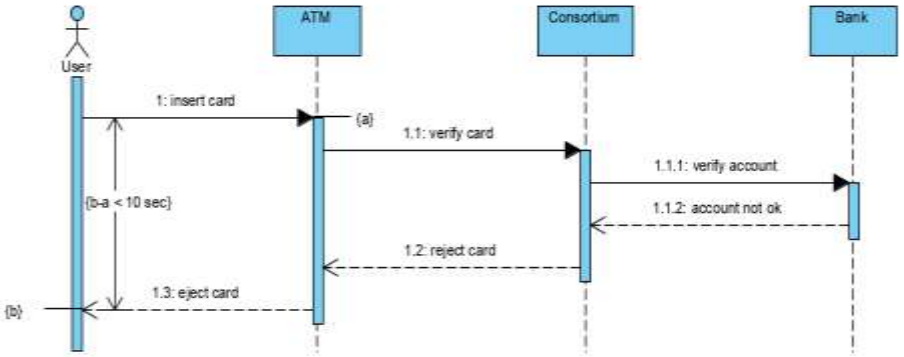
В процессе объектно-ориентированного проектирования определяются логические программные объекты, а также их взаимосвязи и взаимодействие между ними. Эти объекты характеризуются атрибутами и операциями. Например, класс «издание» может содержать атрибут «название» и операцию «выдать».

Диаграмма последовательности *отображает взаимодействие объектов в динамике*.

Диаграмма последовательности относится к диаграммам взаимодействия UML, описывающим поведенческие аспекты системы, но *рассматривает взаимодействие объектов во времени*.

Диаграмма последовательности отображает временные особенности передачи и приема сообщений объектами.





Кооперация (collaboration) служит для обозначения множества взаимодействующих с определенной целью объектов в общем контексте моделируемой системы. Цель самой кооперации состоит в том, чтобы специфицировать особенности реализации отдельных наиболее значимых операций в системе. Кооперация определяет структуру поведения системы в терминах взаимодействия участников этой кооперации.

Кооперация может быть представлена на двух уровнях:

• На уровне спецификации – показывает роли классификаторов и роли ассоциаций в рассматриваемом взаимодействии.

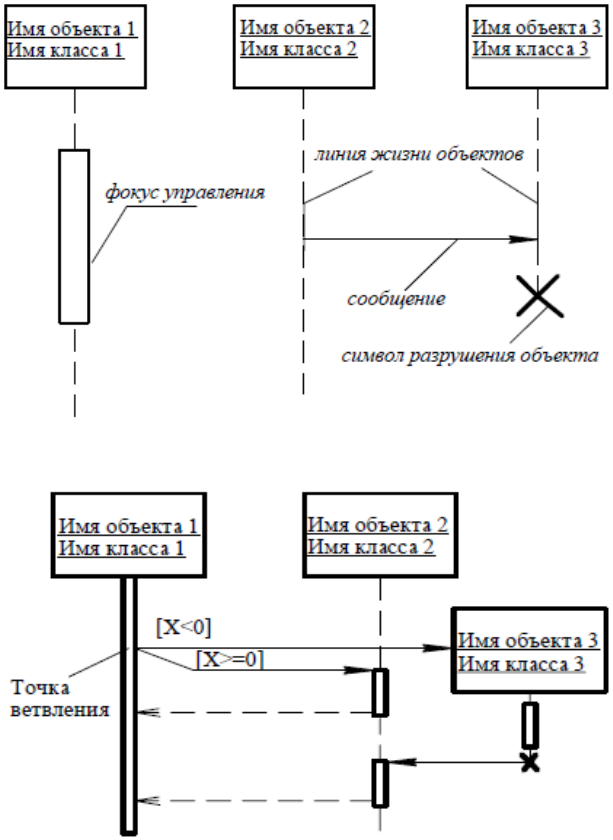
• На уровне примеров – указывает экземпляры и связи, образующие отдельные роли в кооперации.

Сообщения могут быть следующих видов:

– синхронное сообщение (англ. synchronous message). Клиент посылает сообщение серверу и ждет, пока тот примет и обработает сообщение. Как правило, один объект передает синхронное сообщение второму, второй – третьему и т.д., образуя вложенный поток сообщений. В любом случае клиент, инициирующий поток сообщений, должен дождаться его завершения, т.е. возврата управления. Это самый распространенный тип сообщений;

– асинхронное сообщение (англ. asynchronous message). Клиент посылает сообщение серверу и, не дожидаясь ответа, продолжает выполнять следующие операции;
– возвращающее сообщение (англ. reply message), обозначающее возврат значения или управления от сервера обратно клиенту. Стрелки этого вида зачастую отсутствуют на диаграммах, поскольку неявно предполагается их существование после окончания процесса выполнения операции.

В отдельных случаях объект может посылать сообщения самому себе (вызывать собственные методы), инициируя так называемые рефлексивные сообщения.



При разработке диаграмм следует придерживаться следующих правил:

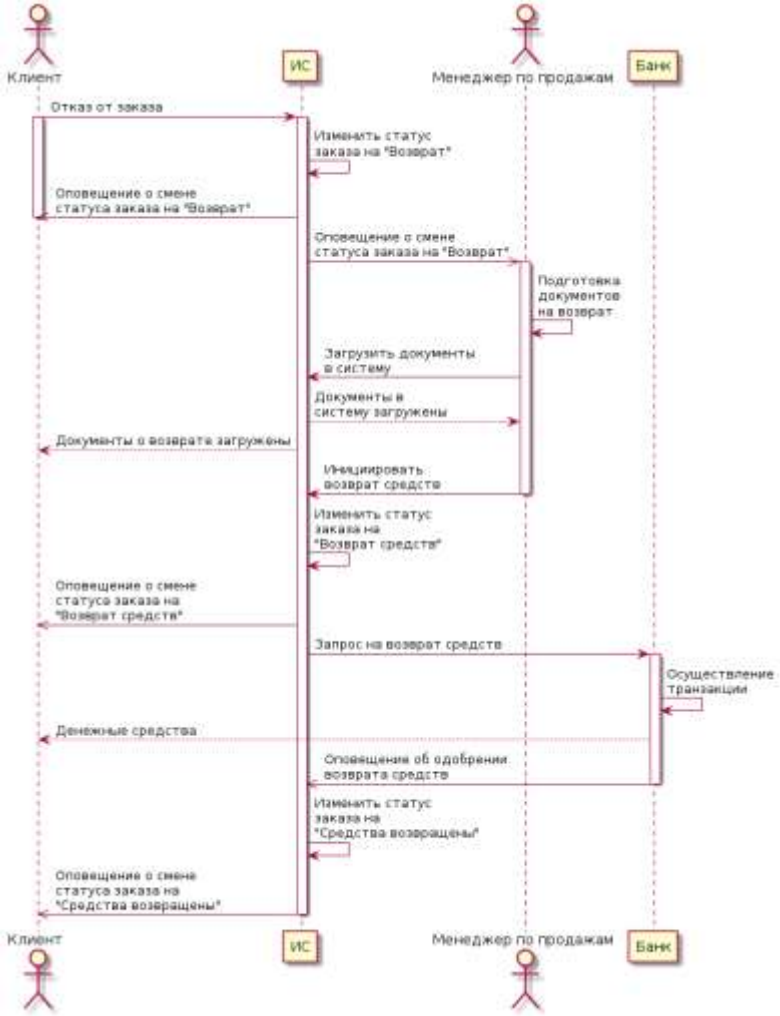
1. Для выбранного варианта использования необходимо перенести с диаграммы классов анализа все участвующие в нем классы, а с диаграммы вариантов использования – актеров.

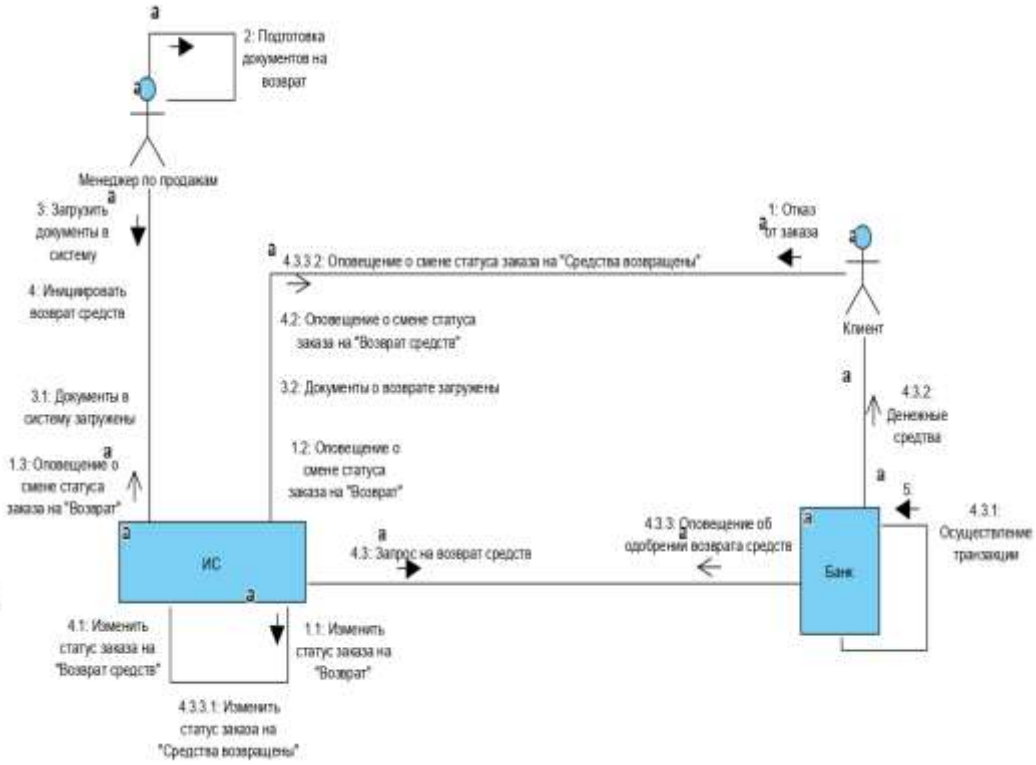
2. На диаграмме коммуникации между классами следует отобразить ассоциации, перенесенные с диаграммы классов анализа, а также добавить ассоциации, связывающие актеров с граничными классами.

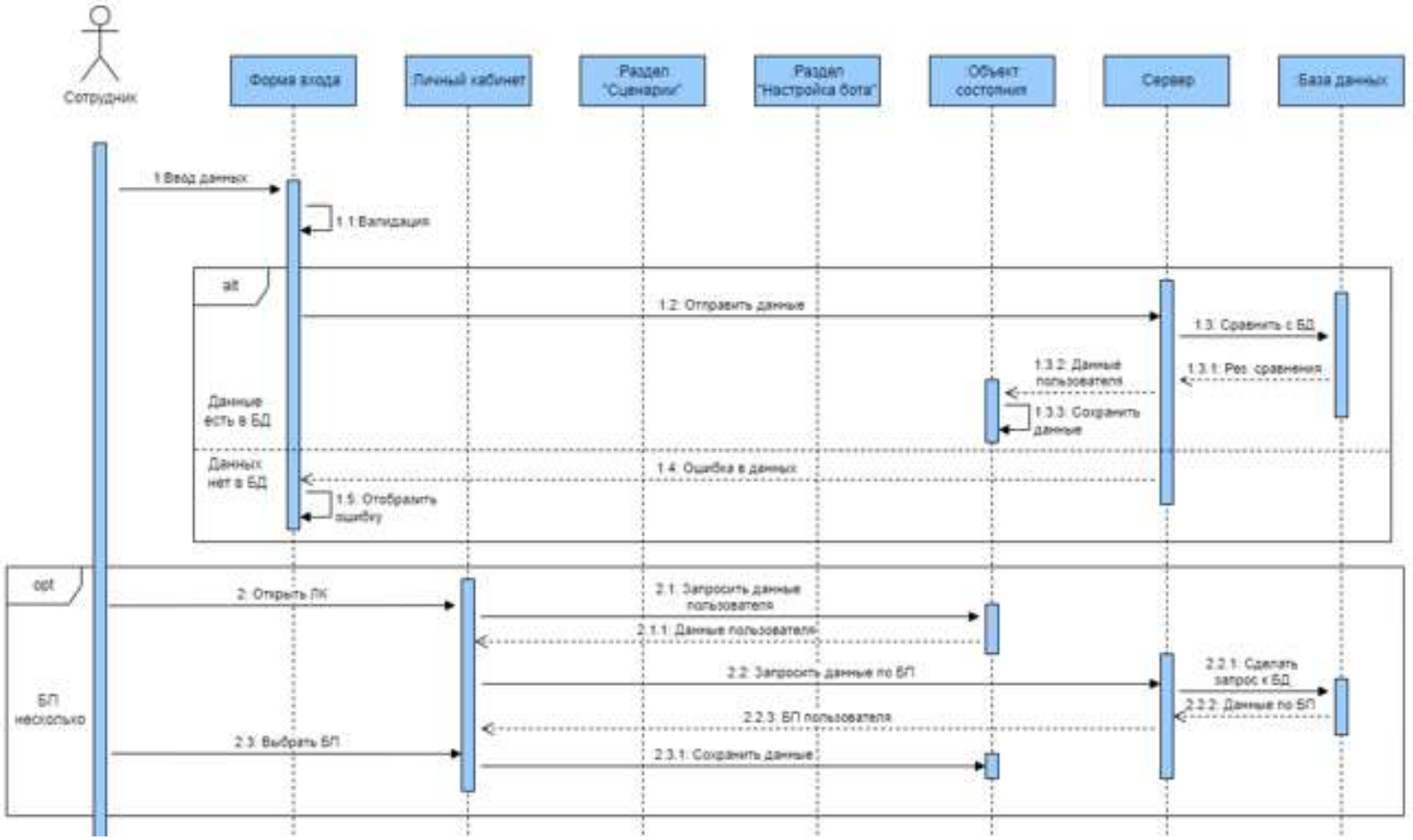
3. На стадии анализа имена сообщениям можно давать произвольно или в виде стереотипов. В дальнейшем (в модели проектирования) имена сообщений должны соответствовать методам классов.

4. Имена сущностей на диаграммах (экземпляры актеров и объекты) должны быть подчеркнуты и обозначены соответствующим образом.

5. На диаграммах последовательности символ уничтожения объектов следует задавать только для тех объектов, которые во время взаимодействия действительно уничтожаются.


Клиент
ИС
Менеджер по продажам
Банк
Отказ от заказа
Изменить статус заказа на "Возврат"
Оповещение о смене статуса заказа на "Возврат"
Оповещение о смене статуса заказа на "Возврат"
Подготовка документов на возврат
Загрузить документы в систему
Документы в систему загружены
Документы о возврате загружены
Инициировать возврат средств
Изменить статус заказа на "Возврат средств"
Оповещение о смене статуса заказа на "Возврат средств"
Запрос на возврат средств
Осуществление транзакции
Денежные средства
Оповещение об одобрении возврата средств
Изменить статус заказа на "Средства возвращены"
Оповещение о смене статуса заказа на "Средства возвращены"
Клиент
ИС
Менеджер по продажам
Банк


2: Подготовка документов на возврат
Менеджер по продажам
3: Загрузить документы в систему
4: Инициировать возврат средств
1: Отказ от заказа
Клиент
4.3.3.2: Оповещение о смене статуса заказа на "Средства возвращены"
4.2: Оповещение о смене статуса заказа на "Возврат средств"
3.2: Документы о возврате загружены
4.3.2 Денежные средства
3.1: Документы в систему загружены
1.2: Оповещение о смене статуса заказа на "Возврат"
1.3: Оповещение о смене статуса заказа на "Возврат"
5
4.3.1 Осуществление транзакции
4.3.3: Оповещение об одобрении возврата средств
ИС
4.3: Запрос на возврат средств
Банк
4.1: Изменить статус заказа на "Возврат средств"
1.1: Изменить статус заказа на "Возврат"
4.3.3.1: Изменить статус заказа на "Средства возвращены"


Сотрудник
Форма входа
Личный кабинет
Раздел "Сценарии"
Раздел "Настройка бота"
Объект состояния
Сервер
База данных
1 Ввод данных
1.1 Валидация
alt
1.2. Отправить данные
1.3. Сравнить с БД
1.3.2. Данные пользователя
1.3.1. Рез. сравнения
1.3.3. Сохранить данные
Данные есть в БД
Данных нет в БД
1.4. Ошибка в данных
1.5. Отобразить ошибку
opt
2. Открыть ЛК
2.1. Запросить данные пользователя
2.1.1. Данные пользователя
2.2. Запросить данные по БП
2.2.1. Сделать запрос к БД
2.2.2. Данные по БП
2.2.3. БП пользователя
БП несколько
2.3. Выбрать БП
2.3.1. Сохранить данные

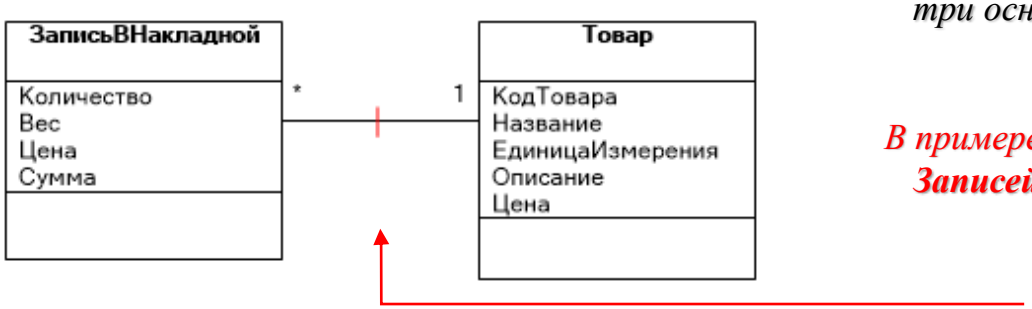
Назначение и состав диаграммы классов

Диаграмма классов определяет типы классов системы и различного рода статические связи, которые существуют между ними. На диаграммах классов изображаются также атрибуты классов, операции классов и ограничения, которые накладываются на связи между классами. Вид и интерпретация диаграммы классов существенно зависит от точки зрения (уровня абстракции): классы могут представлять ***сущности предметной области*** (в процессе анализа) или ***элементы программной системы*** (в процессах проектирования и реализации).



Видимость отображается с помощью следующих символов:
- "+" – общедоступный атрибут (англ. public) – доступен для чтения и модификации из объектов любого класса;
- "#" – защищенный атрибут (англ. protected) – доступен только объектам описываемого класса и его потомкам при наследовании;
- "–" – закрытый атрибут (англ. private) – доступен только объектам описываемого класса;
- "~" – пакетный атрибут (англ. package) – доступен только объектам классов, входящих в тот же пакет.

Видимость *(англ. visibility) характеризует возможность чтения и модификации значения атрибута объекта описываемого класса, из объектов других классов. Модификация значения возможна лишь при условии, что атрибут не является константой.*



Стереотип классов *– это механизм, позволяющий разделять классы на категории. В языке UML определены три основных стереотипа классов: entity, boundary, control.*

В примере на рисунке каждый ***Товар*** *имеет сколь угодно* ***Записей*** *в накладной, но каждая* ***Запись*** *в накладной обязательно один* ***Товар****.*

При разработке диаграммы следует придерживаться следующих правил:

1. За основу диаграммы классов при ее разработке берется диаграмма классов анализа.

2. Для классов должны быть определены и специфицированы все атрибуты и методы. Их спецификация, как правило, выполняется с учетом выбранного языка программирования.

3. При определении методов рекомендуется использовать сообщения с ранее разработанных диаграмм последовательности и коммуникации.

4. Детальное проектирование граничных классов, как правило, не требуется. Большинство современных средств разработки поддерживает визуальную разработку интерфейса системы – меню, диалоговых форм, элементов диалоговых окон, панелей инструментов и т. д. В качестве исходных данных для их проектирования служат прототипы пользовательских интерфейсов. В связи с этим при проектировании таких классов основное внимание следует уделять особенностям отображения информации и специфичным операциям, которые возникают при диалоге пользователя с системой. Граничные классы, определяющие интерфейс взаимодействия с другими системами, требуют детального проектирования.

5. Для проектирования классов-сущностей можно применять подходы, используемые при проектировании БД, особенно в том случае, если данные будут храниться в таблицах БД. Если представление данных в БД и классах отличается друг от друга и в качестве хранилища информации будет применяться реляционная база данных, то рекомендуется разработать отдельную диаграмму классов, описывающую состав и структуру БД.

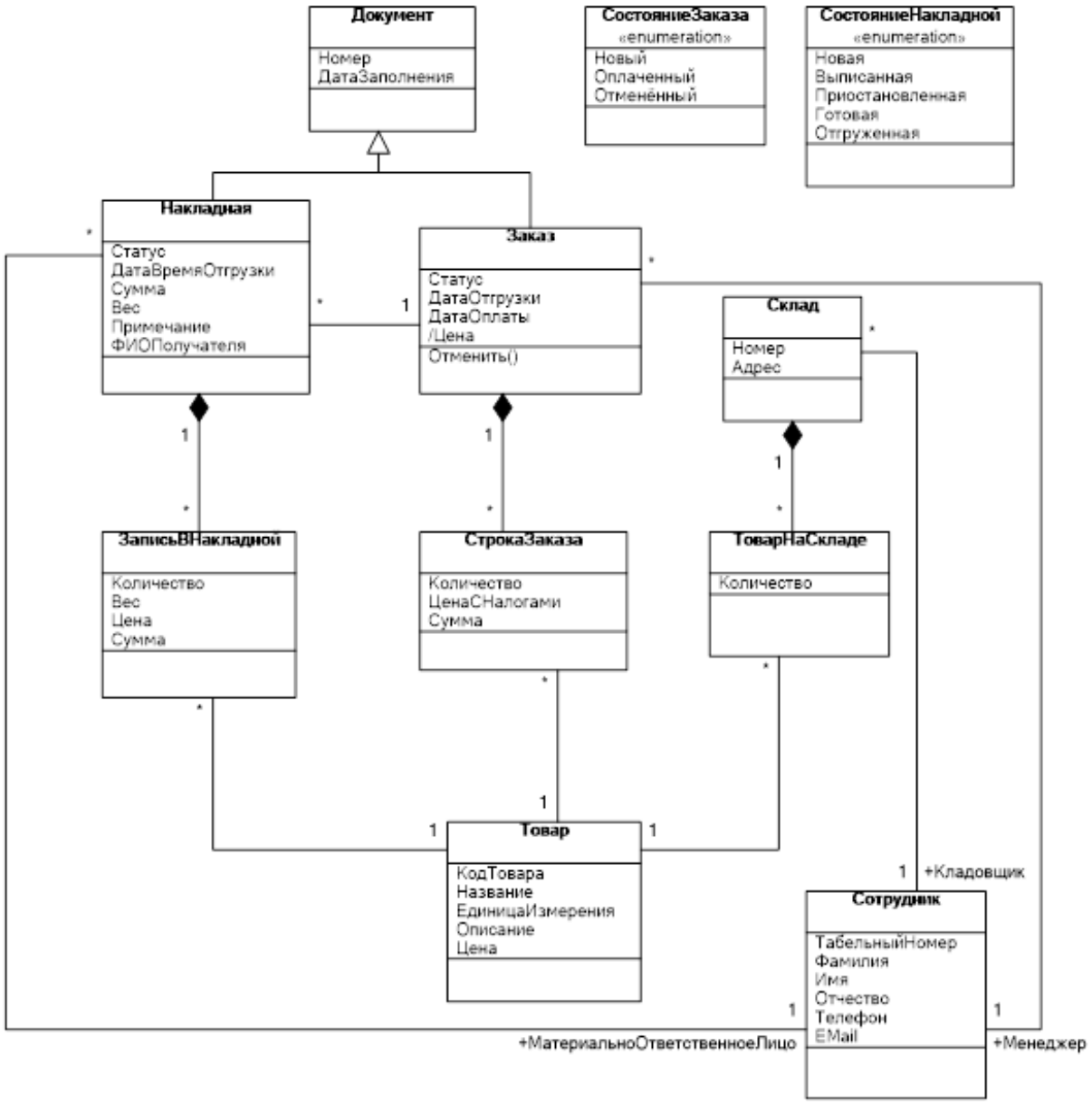
6. Несмотря на то, что каждому объекту при выполнении программы автоматически назначается уникальный идентификатор, рекомендуется для классов-сущностей явно определять атрибуты, хранящие значения первичного ключа.

7. В отличие от реляционных БД поощряется использование в классах многозначных атрибутов в виде массивов, множеств, списков и т. д.

8. Управляющие классы следует проектировать только в случаях крайней необходимости – управления сложным взаимодействием объектов, реализации сложной бизнес-логики и вычислений, контроля целостности объектов и т. п. В противном случае функциональность этого класса лучше распределить между соответствующими граничными классами и классами-сущностями.

В третьей секции указывается перечень операций класса. Можно выделить шесть основных типов операций:

1. ***конструктор*** *– метод, создающий и инициализирующий объект. В Java имя конструктора совпадает с именем класса;*
2. ***деструктор*** *– метод, уничтожающий объект. В некоторых языках программирования (в частности в Java) определение деструкторов не требуется, так как очистка памяти от неиспользуемых объектов (сборка мусора) выполняется автоматически;*
3. ***модификатор*** *– метод, который изменяет состояние объекта (значения атрибутов). Имена модификаторов начинаются, как правило, со слова set (англ. – установить). Например, установить атрибуту Name новое значение setName (newName : String);*
4. ***селектор*** *– метод, который может только считывать значения атрибутов объекта, но не изменяет их. Имена селекторов начинаются, как правило, со слов get (англ. – получить) или is при возврате логического результата. Например, считать значение атрибута Name – getName() или определить видимость на экране элемента графического интерфейса – isVisible();*
5. ***итератор*** *– метод, позволяющий организовать доступ к элементам объекта. Например, для объекта, представляющего собой множество Set или список List, это могут быть методы перехода к первому элементу first(), следующему next(), предыдущему previous() и т. п.;*
6. ***событие*** *– метод, запускаемый на выполнение автоматически при соблюдении определенных условий.*


Документ
Номер
ДатаЗаполнения
СостояниеЗаказа
«enumeration»
Новый
Оплаченный
Отменённый
СостояниеНакладной
«enumeration»
Новая
Выписанная
Приостановленная
Готовая
Отгруженная
Накладная
Статус
ДатаВремяОтгрузки
Сумма
Вес
Примечание
ФИОПолучателя
Заказ
Статус
ДатаОтгрузки
ДатаОплаты
/Цена
Отменить()
Склад
Номер
Адрес
ЗаписьВНакладной
Количество
Вес
Цена
Сумма
СтрокаЗаказа
Количество
ЦенаСНалогами
Сумма
ТоварНаСкладе
Количество
Товар
КодТовара
Название
ЕдиницаИзмерения
Описание
Цена
Сотрудник
ТабельныйНомер
Фамилия
Имя
Отчество
Телефон
EMail
+Кладовщик
+МатериальноОтветственноеЛицо
+Менеджер

Спасибо за внимание!